

## MODUL PRAKTIKUM PERTEMUAN 5

### Implementasi Relasi Database pada REST API Laravel

#### Tujuan Pembelajaran

Setelah mengikuti praktikum ini, mahasiswa mampu:

1. Memahami konsep relasi database dalam sistem client–server.
2. Membuat tabel yang saling berhubungan menggunakan foreign key.
3. Mengimplementasikan Eloquent Relationship pada Laravel.
4. Menghubungkan data antar tabel dalam REST API.
5. Menampilkan data relasi menggunakan endpoint API.

#### Dasar Teori Singkat

##### 1. Relasi Database

Dalam sistem informasi, data sering memiliki hubungan dengan data lain. Hubungan ini disebut relasi database.

Contoh sederhana:

Satu kategori produk dapat memiliki banyak produk.

Contoh tabel:

##### Tabel Categories

| id | name       |
|----|------------|
| 1  | Elektronik |
| 2  | Aksesoris  |

##### Tabel Products

| id | name         | price  | category_id |
|----|--------------|--------|-------------|
| 1  | Mouse Gaming | 250000 | 2           |
| 2  | Keyboard     | 850000 | 2           |

Pada tabel products terdapat field category\_id yang berfungsi sebagai foreign key.

## 2. Jenis Relasi Database

Beberapa jenis relasi database yang umum digunakan:

| Jenis Relasi | Penjelasan                                 |
|--------------|--------------------------------------------|
| One-to-One   | satu data berhubungan dengan satu data     |
| One-to-Many  | satu data berhubungan dengan banyak data   |
| Many-to-Many | banyak data berhubungan dengan banyak data |

Pada praktikum ini akan digunakan relasi:

### One-to-Many

Category → memiliki banyak → Product

## 3. Eloquent Relationship

Laravel menyediakan fitur Eloquent ORM yang memudahkan pengelolaan relasi database.

Contoh relationship:

Category hasMany Product  
Product belongsTo Category

### Alat dan Bahan

1. Laptop / komputer
2. Visual Studio Code
3. Laravel Project dari pertemuan sebelumnya
4. Database MySQL
5. Postman

## Langkah Kerja Praktikum

### A. Membuat Migration Tabel Categories

Jalankan perintah berikut pada terminal.

```
php artisan make:migration create_categories_table
```

### B. Membuat Struktur Tabel categories

Buka file migration yang baru dibuat pada folder:

**database/migrations**

Kemudian ubah kode menjadi:

```
1 public function up()
2 {
3     Schema::create('categories', function (Blueprint $table) {
4         $table->id();
5         $table->string('name');
6         $table->timestamps();
7     });
8 }
```

### C. Menjalankan Migration

Jalankan perintah berikut untuk membuat tabel pada database.

```
php artisan migrate
```

Setelah berhasil maka tabel categories akan muncul di database.

## D. Menambahkan Foreign Key pada Tabel Products

Buat migration baru.

```
php artisan make:migration add_category_id_to_products_table
```

Kemudian ubah kode migration menjadi



```
1 public function up()
2 {
3     Schema::table('products', function (Blueprint $table) {
4         $table->foreignId('category_id')->constrained()->onDelete('cascade');
5     });
6 }
7
```

Jalankan kembali migration

```
php artisan migrate
```

## E. Membuat Model Category

Jalankan perintah berikut.

```
php artisan make:model Category
```

Model akan dibuat pada folder:

**app/Models**

## E. Menambahkan Relationship pada Model

Buka file Category.php :



```
1 public function products()  
2 {  
3     return $this->hasMany(Product::class);  
4 }
```

Buka di file Product.php



```
1 public function category()  
2 {  
3     return $this->belongsTo(Category::class);  
4 }
```

## G. Menampilkan Data Relasi pada API

Buka ProductController

Ubah method index() menjadi:



```
1 public function index()  
2 {  
3     $products = Product::with('category')->get();  
4  
5     return response()->json($products);  
6 }  
7
```

Method tersebut akan menampilkan data produk beserta kategorinya.

## H. Menguji API Menggunakan Postman

### 1 Menambahkan Data Category

Endpoint:

POST /api/categories

Body JSON:



```
1 {  
2   "name": "Aksesoris"  
3 }
```

```

1  {
2      "success": true,
3      "message": "Category berhasil ditambahkan",
4      "data": {
5          "name": "Aksesoris",
6          "updated_at": "2026-03-09T05:15:39.000000Z",
7          "created_at": "2026-03-09T05:15:39.000000Z",
8          "id": 1
9      }
10 }

```

## 2. Menambahkan Data Produk dengan Category

Setelah kategori berhasil dibuat, selanjutnya tambahkan produk yang memiliki kategori.

Method:

**POST**

URL:

**<http://127.0.0.1:8000/api/products>**

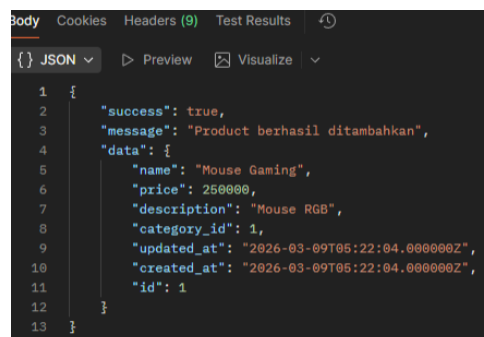
Body JSON:

```

1  {
2      "name": "Mouse Gaming",
3      "price": 250000,
4      "description": "Mouse RGB",
5      "category_id": 1
6  }

```

Jika berhasil maka JSON akan menampilkan :



```

Body Cookies Headers (9) Test Results
({) JSON Preview Visualize
1  {
2      "success": true,
3      "message": "Product berhasil ditambahkan",
4      "data": {
5          "name": "Mouse Gaming",
6          "price": 250000,
7          "description": "Mouse RGB",
8          "category_id": 1,
9          "updated_at": "2026-03-09T05:22:04.000000Z",
10         "created_at": "2026-03-09T05:22:04.000000Z",
11         "id": 1
12     }
13 }

```

### 3. Menampilkan Semua Data Produk

Untuk menampilkan semua produk yang ada di database gunakan endpoint berikut.

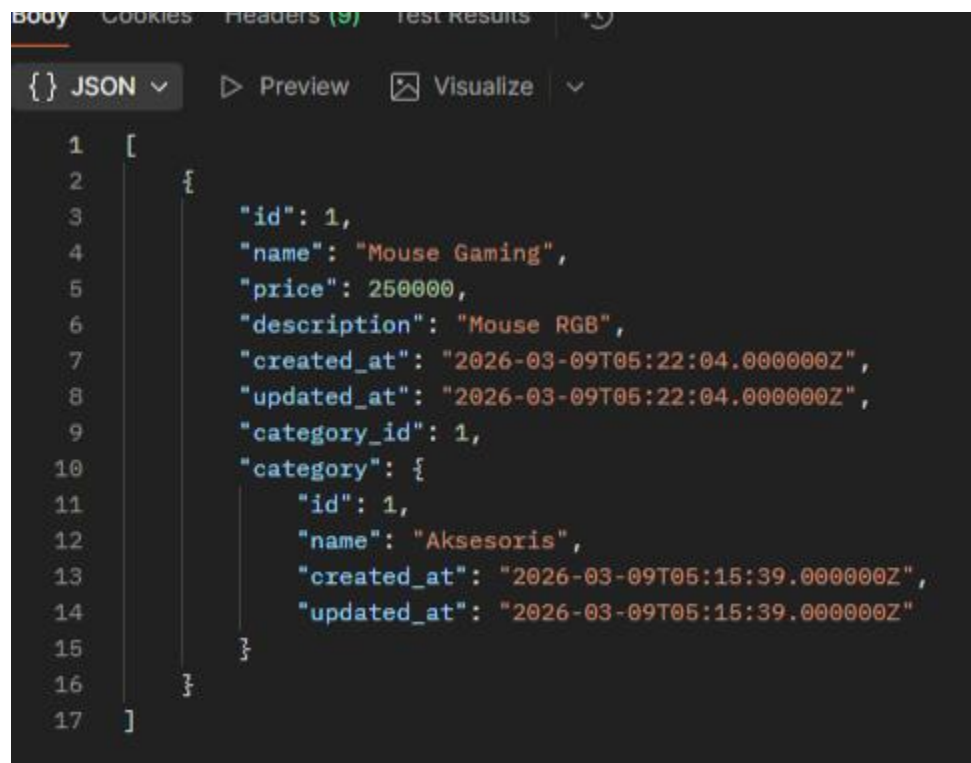
Method:

**GET**

URL:

**<http://127.0.0.1:8000/api/products>**

Jika berhasil maka akan muncul data produk beserta kategori yang dimilikinya.



```
{
  "id": 1,
  "name": "Mouse Gaming",
  "price": 250000,
  "description": "Mouse RGB",
  "created_at": "2026-03-09T05:22:04.000000Z",
  "updated_at": "2026-03-09T05:22:04.000000Z",
  "category_id": 1,
  "category": {
    "id": 1,
    "name": "Aksesoris",
    "created_at": "2026-03-09T05:15:39.000000Z",
    "updated_at": "2026-03-09T05:15:39.000000Z"
  }
}
```

**\*hasil ini berdasarkan percobaan praktikum pertemuan ini yang hanya menambahkan 1 kategori dan 1 produk**



## **I. Latihan**

Kerjakan tugas berikut.

1. Tambahkan minimal 2 kategori produk.
2. Tambahkan minimal 3 produk yang memiliki kategori berbeda.
3. Tampilkan seluruh produk menggunakan endpoint API.
4. Tampilkan satu kategori beserta semua produk yang dimilikinya.

## **J. Diskusi**

Jawab pertanyaan berikut.

1. Mengapa relasi database penting dalam sistem informasi?
2. Apa fungsi foreign key dalam database?
3. Apa keuntungan menggunakan Eloquent Relationship pada Laravel?

## **K. Kesimpulan**

Pada praktikum ini mahasiswa telah mempelajari bagaimana membuat relasi database pada REST API Laravel menggunakan Eloquent Relationship.

Dengan menggunakan relasi database, sistem dapat menghubungkan data antar tabel sehingga pengelolaan informasi menjadi lebih terstruktur dan efisien.

Kemampuan ini merupakan bagian penting dalam pengembangan backend pada sistem client-server modern.